

RETO GEEST

Se necesita un sistema de gestión de trabajo para empresas.

El sistema permite crear tareas y asignarlas a una o varias personas. Cada persona puede marcar su parte como terminada. Cuando todos los usuarios asignados hayan terminado su parte, la tarea se archiva automáticamente y se notifica al sistema del cliente.

Los usuarios pueden tener varias tareas asignadas simultáneamente.

Tu objetivo

Construir una API REST en Node.js (TypeScript es bienvenido) que implemente los endpoints exactos definidos en la sección **Funcionalidad básica**.

Requisitos:

- Utilizar una base de datos SQL real (SQLite, PostgreSQL o MySQL, a elección del candidato).
- Incluir el esquema de la base de datos versionado dentro del repositorio.
- Publicar la API en una URL pública. La evaluación se realizará contra la URL disponible en producción. El proveedor y la forma de despliegue son decisión del candidato y forman parte del reto.
- Entregar el repositorio en GitHub público.
- Incluir tests y README.
- Adjuntar un UML con la estructura de la base de datos, incluyendo tipos de datos y relaciones.

Funcionalidad básica

Para cumplir con el reto, la API deberá contar con los siguientes endpoints:

Endpoint	Comportamiento requerido
POST /users	Body: { "name", "lastName", "email" } Registra un usuario en la base de datos con un ID único. Retorna el ID junto con la información del usuario. Debe devolver un error si falta información obligatoria o si el correo electrónico no es válido.

POST /tasks**Body:** { "title", "description" }

Registra una tarea en la base de datos con un ID único. Su estado por defecto debe ser "open". Retorna el ID de la tarea junto con su información. Debe devolver un error si falta información obligatoria. El título es obligatorio; la descripción es opcional.

**POST
/tasks/:idTask/assign****Body:** { "userIds": [1, 2, 3] }

Asigna el arreglo de usuarios a una tarea. Si algún usuario o la tarea no están registrados, debe devolver un error. Si algún usuario ya está asignado a la tarea, no debe duplicarse la relación. Retorna un mensaje de éxito.

**POST
/tasks/:idTask/complete****Body:** { "userId": 1 }

Marca como completada la parte de la tarea correspondiente al usuario. Si el usuario o la tarea no están registrados, debe devolver un error. Si el usuario no está asignado a la tarea, debe devolver un error. Retorna un mensaje de éxito y marca la participación del usuario en la tarea como completada. Cuando todos los usuarios asignados hayan terminado, la tarea debe cambiar su estado a "archived" y debe enviarse la notificación descrita en la sección **Confiabilidad**.

GET /tasks

Acepta el parámetro opcional `?status=open|archived`. Lista todas las tareas creadas, indicando qué usuarios ya completaron su parte. Si se recibe el parámetro `status`, debe mostrar únicamente las tareas que tengan ese estado.

GET /users

Lista los usuarios registrados, mostrando su información básica y sus tareas pendientes.

GET /users/:idUser/tasks

Lista todas las tareas asignadas al usuario e indica si el usuario ha completado o no su parte en cada una.

GET /tasks/:idTask

Retorna la información completa de la tarea: título, descripción, estado y usuarios asignados, indicando cuáles ya completaron su parte.

**GET
/tasks/:idTask/notifications**

Lista los intentos de envío de notificación correspondientes a esta tarea, según lo descrito en la sección **Confiabilidad**.

Todos los errores deben responder con el siguiente formato:

```
{  
  "error": {
```

```
"code": "...",
"message": "...",
}
}.
```

Confiabilidad

La API debe comportarse correctamente cuando los requests llegan duplicados o de forma concurrente, por ejemplo, debido a un doble clic, reintentos automáticos de otros sistemas o porque dos personas actúan al mismo tiempo.

Se deben cumplir los siguientes requisitos:

1. Idempotencia

Todos los endpoints **POST** deben aceptar el header **Idempotency-Key**.

Si se recibe dos veces un request con el mismo **Idempotency-Key** y el mismo cuerpo, la operación debe ejecutarse una sola vez y ambas respuestas deben ser idénticas.

Esto debe cumplirse incluso cuando ambos requests llegan en paralelo.

2. Archivado sin duplicados

Si los dos últimos usuarios asignados completan la tarea simultáneamente, la tarea debe quedar archivada **exactamente una vez** y la notificación debe enviarse **exactamente una vez**.

3. Notificaciones con reintentos

Cuando una tarea se archive, la API debe realizar un **POST** a una URL externa configurable mediante la variable de entorno **NOTIFY_URL**, enviando:

```
{
  "taskId": 123,
  "title": "Título de la tarea",
  "archivedAt": "2026-08-20T20:00:00Z"
}
```

Si el destino responde con un error **5xx** o no responde, la API debe reintentar el envío con esperas crecientes, hasta un máximo de **3 intentos**.

Cada intento debe quedar registrado, incluyendo:

- número de intento;
- timestamp;

- status HTTP obtenido, cuando exista.

Estos intentos deben poder consultarse mediante `GET /tasks/:idTask/notifications`.

No es necesario desarrollar el sistema externo encargado de mandar las notificaciones.

Unit tests

El repositorio debe incluir tests automatizados que verifiquen la funcionalidad descrita para los endpoints.

El comando necesario para ejecutar los tests debe estar documentado en el README.

Extra

Tu nivel: agrega una mejora que consideres necesaria para este producto y que no esté mencionada anteriormente.

La mejora debe estar **funcionando**, no únicamente documentada o presentada en diapositivas.

Debe ser una sola mejora y no debe afectar la funcionalidad requerida anteriormente.

Explica en el README:

- qué problema resuelve;
- por qué consideraste que era necesaria;
- por qué elegiste esta mejora sobre otras alternativas.

Publicar tu API — requisito obligatorio

La API será evaluada en vivo, por lo que debe estar corriendo en una URL pública accesible desde Internet y mantenerse disponible durante los **7 días posteriores a la entrega**, que corresponde a la ventana de evaluación.

El proveedor de hosting, la forma de despliegue y la base de datos son decisión del candidato. Existen varias alternativas con costo cero o de pocos centavos; encontrar una opción adecuada forma parte del reto.

En el README debes indicar:

- dónde desplegaste la API;
- por qué elegiste esa opción;

- cómo acceder a la API desplegada.

Entregables — checklist final

- URL pública de la API funcionando y disponible durante los 7 días posteriores a la entrega.
- Repositorio en GitHub público.
- SQL del esquema, ya sea mediante script o migraciones, dentro del repositorio.
- UML de la estructura de la base de datos, incluyendo relaciones.
- README de máximo 2 páginas que explique:
 - cómo ejecutar el proyecto localmente, incluyendo los comandos necesarios;
 - las decisiones técnicas importantes y su justificación;
 - los supuestos realizados ante ambigüedades;
 - qué funcionalidades se recortaron por falta de tiempo.

Consideraciones adicionales

Si el documento no especifica algo, no esperes permiso: decide, resuélvelo y documenta tu supuesto en el README.

Tomar decisiones con información incompleta forma parte de lo que evaluamos.

También puedes escribirnos: **las buenas preguntas suman**.

Administra tu alcance. Si algo no te alcanza dentro del plazo, recórtalo y explica el recorte en el README. Preferimos **3 niveles sólidos que 4 a medias**.

Trabaja con las herramientas que utilizas normalmente; más adelante, durante el proceso, deberás defender cada decisión como propia.